ML_F16_SPEEDBRAKE Community Edition Guide

F-16C Speedbrake Position Indicator for DCS World

Build: ML_F16_SPEEDBRAKE_COMMUNITY
Phase: COMMUNITY_EDITION_V1_0
Target hardware: Arduino Nano + 0.96 inch 128x64 I2C SSD1306 OLED
Display library: Adafruit GFX + Adafruit SSD1306
DCS interface: DCS-BIOS / F-16C fork through USB serial
Panel: F-16C SPEED BRAKE Position Indicator

1. Purpose of the Community Edition

The Community Edition is a simplified, working version of the F-16C Speedbrake Position Indicator intended for cockpit builders who want to drive a small OLED display from DCS-BIOS using an Arduino Nano.

It provides a practical three-state indicator: closed, transit, and open. The display is not a flight-certified instrument. It is a hobby simulator component designed for DCS World cockpit builds.

The release follows the same public-release strategy as the ML_F16 Fuel Flow Indicator Community release: useful enough to build, test, and learn from, while keeping advanced framework, productisation, hidden diagnostics, service functions, and commercial support tooling out of the public codebase.

```
CLOSED  = diagonal striped flag
TRANSIT = simple vertical wipe between stripes and dots
OPEN    = 3 x 3 dot flag
```

2. Aircraft and Simulation Reference

The F-16 real-aircraft reference used for cockpit context is the HAF Series Aircraft F-16C/D Blocks 50 and 52+ flight manual. The manual is treated as a layout and systems-context reference only; this public hobby document does not reproduce restricted or operational procedures.

For the DCS implementation, the relevant behaviour is that the F-16C speedbrake position indicator has discrete cockpit indications for the speedbrake state. The Community sketch uses the DCS-BIOS speedbrake raw position output and maps that value into closed, transit, and open display states.

Important: the live-tested community code currently maps CLOSED to stripes and OPEN to dots. Some simulator/manual references describe the opposite visual convention. The safest approach is to preserve the validated DCS mapping for this release, then swap the two renderer calls if a builder needs the opposite pattern for their cockpit reference.

Reference	Use in this document
HAF F-16C/D flight manual	General aircraft/cockpit context for the F-16C/D and speedbrake system reference.
DCS F-16C / DCS-BIOS F-16C fork	Practical data source used by the Arduino sketch.
ML_F16_SPEEDBRAKE_COMMUNITY_V1_0.ino	Authoritative community code for thresholds, state handling, display rendering, and build switches.
ML_F16_SPEEDBRAKE_COMMUNITY_README.md	Condensed release notes, hardware list, library list, and public release boundary.

3. What This Version Includes

Feature	Included
DCS-BIOS live speedbrake input	Yes
Arduino Nano support	Yes
0.96 inch SSD1306 128x64 I2C OLED support	Yes
Adafruit GFX / SSD1306 rendering	Yes
Startup splash	Yes
Standby / waiting for DCS state	Yes
Live display	Yes
Stale-data timeout logic	Yes, freezes last valid cockpit indication by default
Local bench animation without DCS	Yes
Simple raw footer option	Optional compile-time switch
Serial debug	Optional, only when DCS-BIOS is disabled

4. What Has Been Deliberately Removed

The Community Edition deliberately excludes the more advanced project and product features.

Area	Community Edition Position
Pro diagnostics screens	Removed
Hidden service modes	Removed
Advanced panel state framework	Simplified
Protected commercial build logic	Removed
Product support tooling	Removed
Mechanical CAD package	Not included in this code/document release
NVG/dim proxy integration	Not included
Extended raw capture logging	Not included

The aim is to publish a useful working base for builders, not the full internal ML_F16 product framework.

5. Hardware Required

Item	Notes
Arduino Nano	Standard ATmega328P Nano or compatible board.
0.96 inch 128x64 I2C SSD1306 OLED	White OLED used by the current Community sketch.
USB cable	Used for programming and DCS-BIOS serial connection. Must be a data cable, not charge-only.
5V supply	Normally supplied over USB from the DCS-BIOS host PC.
Optional enclosure / cockpit mask	Not included in the Community code release.

The current sketch uses this OLED object:

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

The default I2C address is:

```
#define OLED_ADDR 0x3C
```

6. Basic Wiring

Basic Wiring Diagram

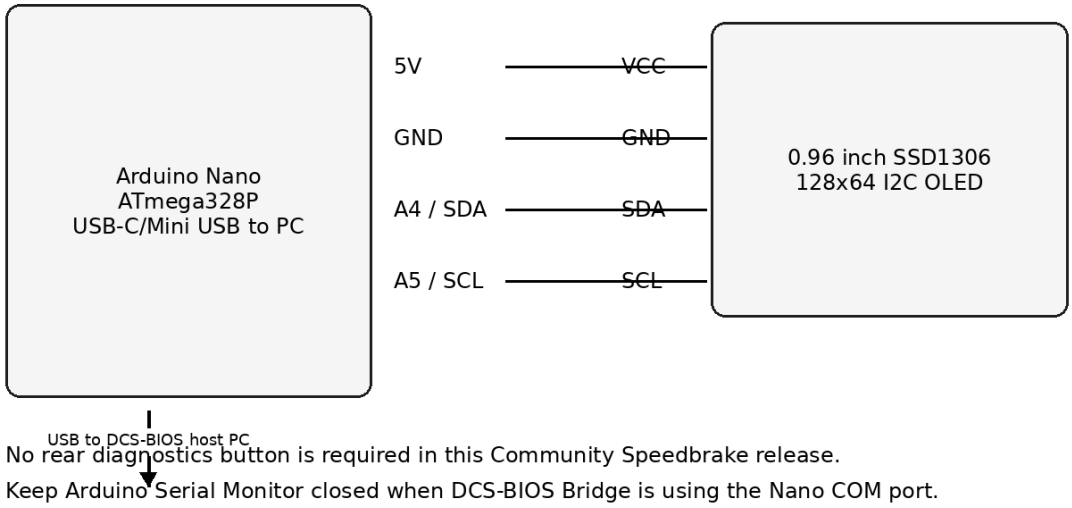


Figure 1 - Basic Arduino Nano to SSD1306 OLED wiring.

OLED Pin	Arduino Nano Pin
VCC	5V or 3.3V, depending on display module. Check your OLED board.
GND	GND
SDA	A4
SCL	A5

Most common 0.96 inch SSD1306 I2C OLED modules work on 5V when connected to an Arduino Nano, but builder modules vary. Check the display module specification before wiring.

There is no mandatory rear diagnostics button in this Community Speedbrake sketch. This is deliberate: the public release is kept small and the display area is better used for the indicator itself.

7. Required Arduino Libraries

Install the following Arduino libraries before compiling the sketch:

Library	Purpose
Wire	I2C support, normally included with Arduino IDE.
Adafruit GFX Library	Graphics primitives used by the SSD1306 renderer.
Adafruit SSD1306	OLED driver for the 128x64 SSD1306 display.
DCS-BIOS library / Flightpanels fork	DCS serial interface and F-16C output definitions.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#if USE_DCS_BIOS
#define DCSBIOS_IRQ_SERIAL
#include "DcsBios.h"
#endif
```

8. DCS-BIOS Requirement

The sketch expects the F-16C DCS-BIOS definitions that expose the speedbrake position output.

Item	Value
Aircraft	F-16C_50
Control	SPEEDBRAKE_INDICATOR
DCS-BIOS macro	F_16C_50_SPEEDBRAKE_INDICATOR
Address	0x44d4
Mask	0xffff
Maximum value	65535

The DCS-BIOS callback in the sketch is:

```
void onSpeedbrakeIndicatorChange(unsigned int newValue) {
  updateDcsChannel(chSpeedbrake, newValue);
  currentSbState = decodeSpeedbrakeState(chSpeedbrake.raw);
}
```

The buffer definition is:

```
DcsBios::IntegerBuffer speedbrakeIndicatorBuffer(
  F_16C_50_SPEEDBRAKE_INDICATOR,
  onSpeedbrakeIndicatorChange
);
```

9. DCS-BIOS Serial Mode

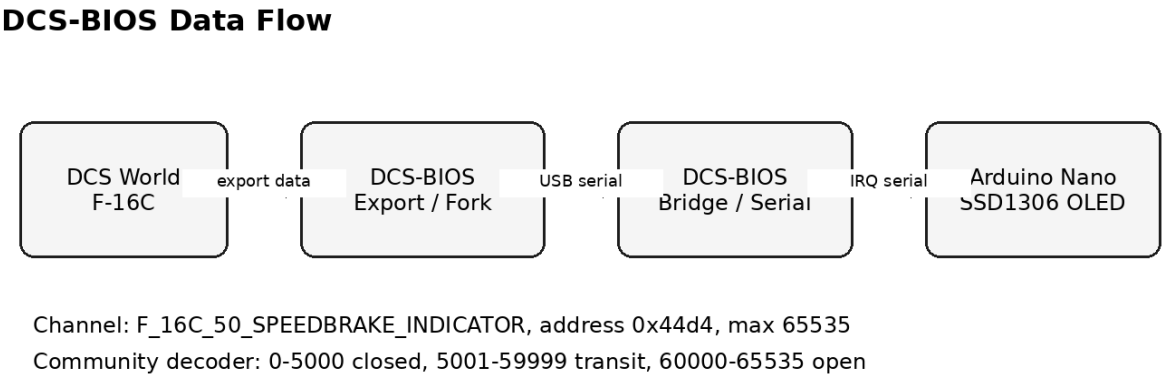


Figure 2 - DCS World to Arduino Nano data flow.

The Community Edition uses DCS-BIOS IRQ serial when live DCS mode is enabled. This is suitable for an Arduino Nano connected directly over USB serial.

In normal use:

- Upload the sketch to the Arduino Nano.
- Connect the Nano to the DCS-BIOS host PC by USB.
- Configure DCS-BIOS Bridge to use the Nano COM port.
- Start DCS World and load the F-16C.
- Once DCS-BIOS begins sending the speedbrake value, the display moves from standby into live operation.

10. Main Configuration Switches

The sketch contains a small set of community-safe compile-time switches.

```
#define USE_DCS_BIOS          1
#define USE_TEST_VALUES       0
#define SHOW_STARTUP_SPLASH   1
#define SHOW_STANDBY_TEXT     1
#define SHOW_RAW_FOOTER       0
#define SHOW_STALE_MARKER     0
#define ENABLE_SERIAL_DEBUG   0
```

10.1 Normal Cockpit Use

```
#define USE_DCS_BIOS     1
#define USE_TEST_VALUES  0
```

10.2 Bench Test Without DCS

```
#define USE_DCS_BIOS     0
#define USE_TEST_VALUES  1
```

Serial debug should only be enabled when DCS-BIOS is disabled. The Arduino Nano uses the same USB serial link for DCS-BIOS IRQ serial, so human-readable serial debug and live DCS-BIOS should not share the port.

11. Display Behaviour

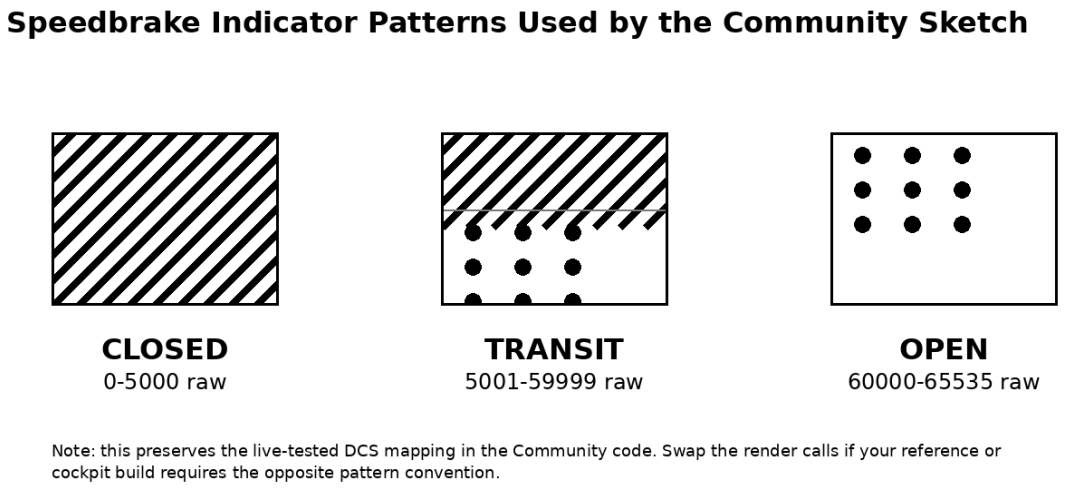


Figure 3 - Indicator patterns and raw threshold bands used by the Community sketch.

11.1 Startup Splash

On power-up, the display shows a simple splash confirming that the OLED and sketch have initialised.

```
ML F-16C SIM
SPEEDBRAKE
COMMUNITY
V1.0
```

11.2 Standby Mode

Before DCS-BIOS data is received, the display enters standby. The default standby message is STBY / NO DCS. This means the Arduino and OLED are running but no live speedbrake data has been seen yet.

11.3 Live Mode

In live mode, the display renders one of the three indicator patterns based on the latest DCS-BIOS raw value.

Raw range	Decoded state	Displayed indication
0 to 5000	SB_CLOSED	Diagonal striped flag
5001 to 59999	SB_TRANSIT	Vertical wipe / drum transition
60000 to 65535	SB_OPEN	3 x 3 dot flag

11.4 Stale Mode

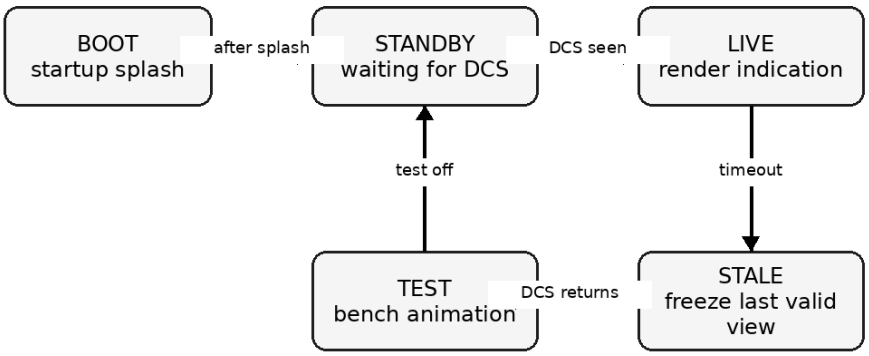
If DCS-BIOS data stops updating for more than the configured timeout, the gauge enters stale mode.

```
#define DCS_TIMEOUT_MS 3000
```

By default the Community release freezes the last valid cockpit indication. A small stale marker can be enabled with SHOW_STALE_MARKER if required, but the clean cockpit display leaves it off.

12. System State Model

Community State Model



Compile-time selection:

```
Live DCS mode: USE_DCS_BIOS=1, USE_TEST_VALUES=0
Bench mode:    USE_DCS_BIOS=0, USE_TEST_VALUES=1
```

Figure 4 - Simplified Community state model.

The Community Edition keeps a reduced state model suitable for a small OLED indicator.

State	Meaning
SYS_BOOT	Startup splash phase.
SYS_STANDBY	Powered but no DCS-BIOS speedbrake data has been received.
SYS_LIVE	Normal live speedbrake display.
SYS_STALE	DCS data has stopped updating. The last valid indication is retained.
SYS_TEST	Local bench animation when USE_TEST_VALUES is enabled.

13. Speedbrake Decoder

The decoder is intentionally simple. It takes the DCS-BIOS raw value and returns one of three public states.

```
SpeedbrakeState decodeSpeedbrakeState(uint16_t raw) {
  if (raw <= SB_CLOSED_MAX_RAW) {
    return SB_CLOSED;
  }

  if (raw >= SB_OPEN_MIN_RAW) {
    return SB_OPEN;
  }

  return SB_TRANSIT;
}
```

The transition percentage is derived from the position between the closed and open thresholds. That percentage is used to split the OLED window between the stripe pattern and the dot pattern.

```
uint8_t speedbrakeTransitionPercent(uint16_t raw) {
  if (raw <= SB_CLOSED_MAX_RAW) return 0;
  if (raw >= SB_OPEN_MIN_RAW) return 100;

  unsigned long span = SB_OPEN_MIN_RAW - SB_CLOSED_MAX_RAW;
  unsigned long pos  = raw - SB_CLOSED_MAX_RAW;
  return (uint8_t)((pos * 100UL) / span);
}
```

14. Indicator Rendering

The OLED indicator area is smaller than the full screen. The Community sketch centres the cockpit indication within a fixed 60 x 44 pixel window.

```
#define IND_X 34
#define IND_Y 10
#define IND_W 60
#define IND_H 44
```

The renderers are deliberately readable: one function draws the closed stripe pattern, one draws the open dot pattern, and one draws the transition. This is easier for community builders to modify than a compact but opaque renderer.

Renderer	Purpose
renderClosedStripes()	Draws the diagonal stripe flag.
renderOpenDots()	Draws the 3 x 3 dot flag.
renderDrumTransition()	Draws a vertical split between stripes and dots based on the raw DCS position.
renderSpeedbrakeState()	Selects the correct renderer for the current decoded state.

15. Local Test Mode

Local test mode allows the OLED and indicator rendering to be tested without DCS World running.

```
#define USE_DCS_BIOS     0
#define USE_TEST_VALUES  1
```

The bench animation approximates the observed DCS movement timings: opening around 3 seconds and closing around 5 seconds.

```
#define TEST_OPEN_STEP_RAW   3500
#define TEST_CLOSE_STEP_RAW  2100
```

Use this mode to check OLED wiring, display orientation, indicator placement, enclosure aperture alignment, and pattern visibility. Return to live DCS mode after bench testing.

16. Upload Procedure

- Open ML_F16_SPEEDBRAKE_COMMUNITY_V1_0.ino in the Arduino IDE.
- Select board: Arduino Nano.
- Select the correct processor option. Many clone Nanos require ATmega328P Old Bootloader.
- Select the correct COM port.
- Install the required libraries.
- Compile the sketch.
- Upload to the Nano.
- Start DCS-BIOS Bridge and select the Nano serial port.
- Start DCS World and load the F-16C.
- Confirm the display moves from STBY / NO DCS to the live speedbrake indication.

17. Recommended Arduino IDE Settings

Setting	Value
Board	Arduino Nano
Processor	ATmega328P or ATmega328P Old Bootloader
Port	Your Nano COM port
Programmer	AVRISP mkII, default is usually fine

If upload fails, try switching between ATmega328P and ATmega328P Old Bootloader.

18. DCS-BIOS / DCS-BIOS Bridge Notes

The recommended connection path is:

```
DCS World -> DCS-BIOS F-16C export -> DCS-BIOS Bridge -> Arduino Nano USB serial -> SSD1306 OLED
```

If the display stays in standby, the Arduino is running but no valid DCS-BIOS speedbrake data is being received. Check the DCS-BIOS installation, DCS-BIOS Bridge COM port, DCS aircraft selection, and that the Arduino Serial Monitor is closed.

19. Troubleshooting

19.1 OLED Blank

- Check OLED VCC and GND.
- Check SDA to A4 and SCL to A5.
- Check the OLED I2C address. The sketch defaults to 0x3C.
- Run an Adafruit SSD1306 example sketch before testing DCS-BIOS.

19.2 Upload Fails

- Check the correct COM port.
- Use a data-capable USB cable.
- Close DCS-BIOS Bridge and Arduino Serial Monitor during upload.
- Try the Old Bootloader processor option.

19.3 Display Shows Splash But Never Goes Live

- Check DCS-BIOS is installed and exporting.
- Check DCS-BIOS Bridge is running.
- Check the correct Nano COM port is selected.
- Load the F-16C in DCS.
- Confirm the F_16C_50_SPEEDBRAKE_INDICATOR macro exists in your DCS-BIOS fork.

19.4 Pattern Appears Reversed

This is the main practical risk in a public release. The current sketch preserves the live-tested mapping from the validated build: closed equals stripes and open equals dots. If your cockpit reference or personal build expects the opposite convention, swap the render calls for SB_CLOSED and SB_OPEN in renderSpeedbrakeState().

```
case SB_CLOSED:
  renderClosedStripes(stale);
  break;

case SB_OPEN:
  renderOpenDots(stale);
  break;
```

20. Code Structure Overview

Section	Purpose
User configuration	Live DCS mode, bench mode, splash, footer, stale marker, serial debug.
Build info	Community build identity.
Display configuration	OLED size, I2C address, indicator window.
Timing	Startup, timeout, bench animation, serial debug interval.
Thresholds	Closed and open raw-value bands.
DCS callback	Receives live DCS-BIOS speedbrake raw values.
Decoder	Maps raw value to closed, transit, or open.
Renderers	Draws stripes, dots, transition, standby, and splash.
Test mode	Runs local animation without DCS.
Setup/loop	Arduino lifecycle.

21. Safe Areas for Community Modification

Area	Example
OLED address	Change OLED_ADDR if your display uses 0x3D.
Indicator window	Adjust IND_X, IND_Y, IND_W, IND_H for your aperture.
Splash text	Personalise the startup screen.
Timeout value	Adjust DCS_TIMEOUT_MS if stale detection is too sensitive.
Raw thresholds	Tune SB_CLOSED_MAX_RAW or SB_OPEN_MIN_RAW if your DCS-BIOS fork differs.
Bench animation speed	Adjust TEST_OPEN_STEP_RAW and TEST_CLOSE_STEP_RAW.

22. Areas Not Included in the Community Edition

- Full product build-control framework
- Advanced diagnostics pages
- Hidden service-mode workflow
- NVG/dim mode integration
- DCS lighting proxy integration
- Extended serial raw capture
- Commercial support routines
- Protected mechanical CAD package
- Pro display variants or future product features

These omissions are intentional. The Community Edition should be treated as a practical public learning build, not the full internal/professional branch.

23. Community Release Notes

Release text:

This is a Community Edition F-16C Speedbrake Position Indicator sketch for DCS World. It is designed for an Arduino Nano and 0.96 inch 128x64 I2C SSD1306 OLED using Adafruit GFX, Adafruit SSD1306, and DCS-BIOS. The display shows a simple cockpit-style speedbrake position indicator with closed, transition, and open states based on the F_16C_50_SPEEDBRAKE_INDICATOR DCS-BIOS output.

This release includes a startup splash, standby state, live DCS-BIOS input, stale timeout handling, a bench animation mode, and simple public rendering logic. Advanced commercial/prototype features have deliberately been removed from this Community Edition.

24. Disclaimer

This project is a hobby cockpit-simulator component for use with DCS World. It is not affiliated with, endorsed by, or approved by Eagle Dynamics, Lockheed Martin, the United States Air Force, the Hellenic Air Force, or any aircraft manufacturer/operator. It is intended for simulation and educational use only.

25. Licence Position

Asset	Suggested position
Code	Custom non-commercial hobby-use licence, or another licence you explicitly choose before publication.
Documentation	Creative Commons Attribution-NonCommercial is a sensible default if you want public reuse but not resale.
CAD/STL files	Separate licence if released later. Not part of this Community code package.
Advanced/pro version	Keep private unless you decide on a commercial licensing model.

26. Quick Start Summary

- Build the circuit: Arduino Nano plus SSD1306 I2C OLED.
- Install Adafruit GFX, Adafruit SSD1306, Wire, and DCS-BIOS.
- Open the Community Edition sketch.
- For bench test, set USE_DCS_BIOS to 0 and USE_TEST_VALUES to 1.
- For live cockpit use, set USE_DCS_BIOS to 1 and USE_TEST_VALUES to 0.
- Upload to the Nano.
- Connect DCS-BIOS Bridge to the Nano COM port.
- Start DCS World and load the F-16C.
- Confirm the display moves from standby to a live speedbrake indication.
- If the pattern convention is wrong for your cockpit, swap closed/open render calls.

27. Final Notes

This Speedbrake module is a strong Community Edition candidate because the core behaviour is simple, useful, and easy to understand. The release gives builders a real working DCS-BIOS indicator while exposing very little protected ML_F16 IP.

The main item to be honest about in public release notes is the visual convention risk: builder references may differ on whether stripes or dots represent closed/open. The code is clear enough that a builder can reverse that mapping safely if their reference requires it.

Recommended release package: INO file, this guide, README, wiring diagram, simple enclosure photo when available, and a short demo video showing closed, transit, and open states in DCS.

28. Appendices

Appendix A - Community README Content Incorporated

ML_F16_SPEEDBRAKE_COMMUNITY

Community Edition speedbrake position indicator for the DCS F-16C.

Purpose

This release gives builders a simple working base for an F-16C speedbrake indicator using DCS-BIOS, an Arduino Nano, and a 0.96 inch SSD1306 OLED.

It follows the same release strategy as the ML_F16 Fuel Flow Indicator Community release: useful enough to build and learn from, but stripped of Pro-only framework features, hidden service functions, extended diagnostics, and commercial support tooling.

Hardware

- Arduino Nano or compatible ATmega328P board
- 0.96 inch 128x64 I2C SSD1306 OLED
- USB serial connection to DCS-BIOS

Arduino libraries

Install these through the Arduino Library Manager:

- Adafruit GFX Library
- Adafruit SSD1306
- DCS-BIOS library / DCS-BIOS Flightpanels fork as used by your setup

DCS-BIOS output

- Aircraft: F-16C_50
- Control: SPEEDBRAKE_INDICATOR
- Address: 0x44d4
- Max value: 65535

Behaviour

- CLOSED: diagonal striped flag
- TRANSIT: simple vertical wipe between stripes and dots
- OPEN: 3 x 3 dot flag

Thresholds

- 0 to 5000: CLOSED
- 5001 to 59999: TRANSIT

- 60000 to 65535: OPEN

Modes

Default live DCS mode:

define USE_DCS_BIOS 1

define USE_TEST_VALUES 0

Bench test mode without DCS:

define USE_DCS_BIOS 0

define USE_TEST_VALUES 1

Serial debug should only be enabled when DCS-BIOS is disabled, because the Nano uses the same USB serial port for DCS-BIOS IRQ serial.

Community release boundary

Included:

- DCS-BIOS live input
- Simple Nano / SSD1306 display rendering
- Basic startup splash
- Standby / waiting for DCS state
- Stale DCS timeout logic
- Bench animation mode

Not included:

- Pro diagnostics screens
- Hidden service modes
- Advanced panel state framework
- Protected commercial build logic
- Product support tooling
- Mechanical CAD package

Appendix B - Community Code

The full Community Edition sketch is included below for release packaging convenience. The separate .ino file remains the source of truth for compiling and upload.

```
/*****************************************************************
 * ML_F16_SPEEDBRAKE_COMMUNITY
 * COMMUNITY_EDITION_V1_0_NANO_SSD1306_DCS_BIOS
 *
```

```
* F-16C Speedbrake Position Indicator - Community Edition
*
* Purpose:
*   A simple, buildable DCS-BIOS driven speedbrake indicator for hobby use.
*   This version is intentionally kept small and readable for an Arduino
*   Nano plus a 0.96 inch 128x64 I2C SSD1306 OLED.
*
* Hardware:
*   - Arduino Nano or compatible ATmega328P board
*   - 0.96 inch 128x64 I2C SSD1306 OLED
*   - DCS-BIOS serial connection over USB
*
* Display behaviour:
*   - CLOSED  = diagonal striped flag
*   - TRANSIT = simple vertical wipe between stripes and dots
*   - OPEN    = 3 x 3 dot flag
*
* DCS-BIOS source:
*   F-16C_50/SPEEDBRAKE_INDICATOR
*   Address: 0x44d4
*   Mask: 0xffff
*   Max Value: 65535
*
* Community thresholds:
*   - 0 to 5000       = CLOSED
*   - 5001 to 59999   = TRANSIT
*   - 60000 to 65535  = OPEN
*
* Notes:
*   - This is the Community release. It deliberately omits Pro framework
*     features such as extended service screens, advanced diagnostics,
*     hidden modes, commercial support tooling, and protected build logic.
*   - Serial diagnostics are disabled when DCS-BIOS is enabled because both
*     use the Nano serial port.
*   - For bench testing without DCS, set USE_DCS_BIOS to 0 and
*     USE_TEST_VALUES to 1.
*****************************************************************/
```

```
// ---------------------------------------------------------------------
// USER CONFIGURATION
// ---------------------------------------------------------------------

#define USE_DCS_BIOS          1
#define USE_TEST_VALUES       0

#define SHOW_STARTUP_SPLASH   1
#define SHOW_STANDBY_TEXT     1
#define SHOW_RAW_FOOTER       0
#define SHOW_STALE_MARKER     0

// Enable only when USE_DCS_BIOS is 0.
// The Arduino Nano cannot use the same USB serial link for both
// DCS-BIOS IRQ serial and human-readable diagnostics.
#define ENABLE_SERIAL_DEBUG   0

// ---------------------------------------------------------------------
// BUILD INFO
// ---------------------------------------------------------------------

#define BUILD_NAME            "ML_F16_SPEEDBRAKE_COMMUNITY"
#define BUILD_VERSION         "COMMUNITY_V1_0"
#define BUILD_DATE            "2026-06-06"
#define PANEL_ID              "F16_SPEEDBRAKE"

// ---------------------------------------------------------------------
// DISPLAY CONFIGURATION
// ---------------------------------------------------------------------

#define SCREEN_WIDTH          128
#define SCREEN_HEIGHT          64
#define OLED_RESET             -1
#define OLED_ADDR            0x3C

#define IND_X                  34
```

```
#define IND_Y                  10
#define IND_W                  60
#define IND_H                  44

// ---------------------------------------------------------------------
// TIMING
// ---------------------------------------------------------------------

#define STARTUP_SPLASH_MS     1500
#define DCS_TIMEOUT_MS        3000
#define TEST_FRAME_MS           80
#define TEST_ENDPOINT_HOLD_MS 1200
#define SERIAL_DEBUG_MS       1000

// ---------------------------------------------------------------------
// SPEEDBRAKE RAW THRESHOLDS
// ---------------------------------------------------------------------

#define SB_CLOSED_MAX_RAW      5000
#define SB_OPEN_MIN_RAW       60000

// Bench animation only. These approximate the observed DCS movement:
// open around 3 seconds, close around 5 seconds.
#define TEST_OPEN_STEP_RAW     3500
#define TEST_CLOSE_STEP_RAW    2100

// ---------------------------------------------------------------------
// LIBRARIES
// ---------------------------------------------------------------------

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#if USE_DCS_BIOS
#define DCSBIOS_IRQ_SERIAL
```

```
#include "DcsBios.h"
#endif

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// ---------------------------------------------------------------
// STATE MODEL
// ---------------------------------------------------------------

enum SystemState {
  SYS_BOOT,
  SYS_STANDBY,
  SYS_LIVE,
  SYS_STALE,
  SYS_TEST
};

enum SpeedbrakeState {
  SB_CLOSED,
  SB_TRANSIT,
  SB_OPEN
};

struct DcsChannel {
  uint16_t raw;
  unsigned long lastUpdateMs;
  uint32_t updateCount;
  bool live;
};

SystemState systemState = SYS_BOOT;
SpeedbrakeState currentSbState = SB_CLOSED;
SpeedbrakeState lastRenderedSbState = SB_TRANSIT;

DcsChannel chSpeedbrake = {
  0,
  0,
```

```
  0,
  false
};

bool dcsEverSeen = false;
unsigned long lastDcsPacketMs = 0;
unsigned long lastSerialDebugMs = 0;

bool lastRenderedStandby = false;
bool lastRenderedStale = false;
uint8_t lastRenderedTransitionPercent = 255;

// ---------------------------------------------------------------------
// DECODING
// ---------------------------------------------------------------------

SpeedbrakeState decodeSpeedbrakeState(uint16_t raw) {
  if (raw <= SB_CLOSED_MAX_RAW) {
    return SB_CLOSED;
  }

  if (raw >= SB_OPEN_MIN_RAW) {
    return SB_OPEN;
  }

  return SB_TRANSIT;
}

uint8_t speedbrakeTransitionPercent(uint16_t raw) {
  if (raw <= SB_CLOSED_MAX_RAW) {
    return 0;
  }

  if (raw >= SB_OPEN_MIN_RAW) {
    return 100;
  }
```

```
  unsigned long span = SB_OPEN_MIN_RAW - SB_CLOSED_MAX_RAW;
  unsigned long pos = raw - SB_CLOSED_MAX_RAW;

  return (uint8_t)((pos * 100UL) / span);
}

const char* speedbrakeStateName(SpeedbrakeState state) {
  switch (state) {
    case SB_CLOSED:
      return "CLOSED";
    case SB_TRANSIT:
      return "TRANSIT";
    case SB_OPEN:
      return "OPEN";
    default:
      return "UNKNOWN";
  }
}

const char* systemStateName(SystemState state) {
  switch (state) {
    case SYS_BOOT:
      return "BOOT";
    case SYS_STANDBY:
      return "STANDBY";
    case SYS_LIVE:
      return "LIVE";
    case SYS_STALE:
      return "STALE";
    case SYS_TEST:
      return "TEST";
    default:
      return "UNKNOWN";
  }
}
```

```
// ---------------------------------------------------------------------
// DCS-BIOS
// ---------------------------------------------------------------------

void updateDcsChannel(DcsChannel& ch, uint16_t newValue) {
  ch.raw = newValue;
  ch.lastUpdateMs = millis();
  ch.updateCount++;
  ch.live = true;

  lastDcsPacketMs = millis();
  dcsEverSeen = true;
}

void onSpeedbrakeIndicatorChange(unsigned int newValue) {
  updateDcsChannel(chSpeedbrake, (uint16_t)newValue);
  currentSbState = decodeSpeedbrakeState(chSpeedbrake.raw);
}

#if USE_DCS_BIOS
DcsBios::IntegerBuffer speedbrakeIndicatorBuffer(
  F_16C_50_SPEEDBRAKE_INDICATOR,
  onSpeedbrakeIndicatorChange
);
#endif

// ---------------------------------------------------------------------
// DISPLAY HELPERS
// ---------------------------------------------------------------------

void clearDisplayBlack() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
}

void drawCenteredText(const char* text, uint8_t textSize, int y) {
  int16_t x1, y1;
```

```
  uint16_t w, h;

  display.setTextSize(textSize);
  display.getTextBounds(text, 0, y, &x1, &y1, &w, &h);

  int x = (SCREEN_WIDTH - w) / 2;
  display.setCursor(x, y);
  display.print(text);
}

bool pointInIndicatorWindow(int x, int y) {
  return (x >= IND_X &&
          x < IND_X + IND_W &&
          y >= IND_Y &&
          y < IND_Y + IND_H);
}

void drawRawFooter() {
#if SHOW_RAW_FOOTER
  display.fillRect(0, 54, 128, 10, SSD1306_BLACK);
  display.setTextSize(1);
  display.setCursor(0, 56);
  display.print("RAW:");
  display.print(chSpeedbrake.raw);
#endif
}

void drawStaleMarker() {
#if SHOW_STALE_MARKER
  display.setTextSize(1);
  display.setCursor(104, 56);
  display.print("STL");
#endif
}

// ---------------------------------------------------------------------
// FLAG DRAWING
```

```
// ---------------------------------------------------------------------


void drawStripePatternInWindow(int yOffset = 0) {
  for (int x = IND_X - IND_H - 20;
       x < IND_X + IND_W + IND_H + 20;
       x += 12) {


    for (int t = 0; t < 4; t++) {
      int x0 = x + t;
      int y0 = IND_Y + IND_H + yOffset;


      for (int i = 0; i <= IND_H + 12; i++) {
        int px = x0 + i;
        int py = y0 - i;


        if (pointInIndicatorWindow(px, py)) {
          display.drawPixel(px, py, SSD1306_WHITE);
        }
      }
    }
  }
}


void drawDotPatternInWindow() {
  const int startX = IND_X + 8;
  const int startY = IND_Y + 6;
  const int gapX = 22;
  const int gapY = 16;
  const int radius = 4;


  for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
      int x = startX + (col * gapX);
      int y = startY + (row * gapY);
      display.fillCircle(x, y, radius, SSD1306_WHITE);
    }
  }
```

```
}

void drawStripePatternClippedAboveY(int visibleToY, int yOffset = 0) {
  for (int x = IND_X - IND_H - 20;
       x < IND_X + IND_W + IND_H + 20;
       x += 12) {

    for (int t = 0; t < 4; t++) {
      int x0 = x + t;
      int y0 = IND_Y + IND_H + yOffset;

      for (int i = 0; i <= IND_H + 12; i++) {
        int px = x0 + i;
        int py = y0 - i;

        if (pointInIndicatorWindow(px, py) && py <= visibleToY) {
          display.drawPixel(px, py, SSD1306_WHITE);
        }
      }
    }
  }
}

void drawDotPatternClippedBelowY(int visibleFromY) {
  const int startX = IND_X + 8;
  const int startY = IND_Y + 6;
  const int gapX = 22;
  const int gapY = 16;
  const int radius = 4;

  for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
      int cx = startX + (col * gapX);
      int cy = startY + (row * gapY);

      for (int y = cy - radius; y <= cy + radius; y++) {
        for (int x = cx - radius; x <= cx + radius; x++) {
```

```
          int dx = x - cx;
          int dy = y - cy;

          if ((dx * dx + dy * dy) <= (radius * radius) &&
              pointInIndicatorWindow(x, y) &&
              y >= visibleFromY) {
            display.drawPixel(x, y, SSD1306_WHITE);
          }
        }
      }
    }
  }
}

// ---------------------------------------------------------------------
// RENDERING
// ---------------------------------------------------------------------

void renderStartupSplash() {
  clearDisplayBlack();

  display.setTextSize(1);
  display.setCursor(8, 6);
  display.print("ML F-16C SIM");

  display.setCursor(8, 19);
  display.print("SPEEDBRAKE");

  display.setCursor(8, 34);
  display.print("COMMUNITY V1.0");

  display.setCursor(8, 48);
  display.print("NANO SSD1306");

  display.display();
}
```

```
void renderStandby() {
  clearDisplayBlack();

#if SHOW_STANDBY_TEXT
  drawCenteredText("STBY", 2, 18);
  drawCenteredText("WAIT DCS", 1, 42);
#endif

  display.display();

  lastRenderedStandby = true;
  lastRenderedStale = false;
}

void renderClosedStripes(bool stale = false) {
  clearDisplayBlack();

  drawStripePatternInWindow();
  drawRawFooter();

  if (stale) {
    drawStaleMarker();
  }

  display.display();

  lastRenderedStandby = false;
  lastRenderedStale = stale;
  lastRenderedTransitionPercent = 0;
}

void renderOpenDots(bool stale = false) {
  clearDisplayBlack();

  drawDotPatternInWindow();
```

```
  drawRawFooter();

  if (stale) {
    drawStaleMarker();
  }

  display.display();

  lastRenderedStandby = false;
  lastRenderedStale = stale;
  lastRenderedTransitionPercent = 100;
}

void renderDrumTransition(uint8_t percent, bool stale = false) {
  clearDisplayBlack();

  int splitY = IND_Y + IND_H - ((IND_H * percent) / 100);
  int stripeYOffset = (percent / 8) % 12;

  drawStripePatternClippedAboveY(splitY, stripeYOffset);
  drawDotPatternClippedBelowY(splitY);

  drawRawFooter();

  if (stale) {
    drawStaleMarker();
  }

  display.display();

  lastRenderedStandby = false;
  lastRenderedStale = stale;
  lastRenderedTransitionPercent = percent;
}

void renderSpeedbrakeState(bool force = false, bool stale = false) {
```

```
  uint8_t transitionPercent = speedbrakeTransitionPercent(chSpeedbrake.raw);

  if (!force &&
      currentSbState == lastRenderedSbState &&
      lastRenderedStale == stale &&
      lastRenderedTransitionPercent == transitionPercent &&
      !lastRenderedStandby) {
    return;
  }

  lastRenderedSbState = currentSbState;

  switch (currentSbState) {
    case SB_CLOSED:
      renderClosedStripes(stale);
      break;

    case SB_TRANSIT:
      renderDrumTransition(transitionPercent, stale);
      break;

    case SB_OPEN:
      renderOpenDots(stale);
      break;

    default:
      renderClosedStripes(stale);
      break;
  }
}

// ---------------------------------------------------------------------
// SYSTEM STATE
// ---------------------------------------------------------------------

void updateSystemState() {
```

```
#if USE_TEST_VALUES
  systemState = SYS_TEST;
  return;
#endif

  if (!dcsEverSeen) {
    systemState = SYS_STANDBY;
    return;
  }

  if ((millis() - lastDcsPacketMs) > DCS_TIMEOUT_MS) {
    systemState = SYS_STALE;
    return;
  }

  systemState = SYS_LIVE;
}

void renderCurrentSystemState() {
  switch (systemState) {
    case SYS_BOOT:
      renderStartupSplash();
      break;

    case SYS_STANDBY:
      if (!lastRenderedStandby) {
        renderStandby();
      }
      break;

    case SYS_LIVE:
      renderSpeedbrakeState(false, false);
      break;

    case SYS_STALE:
#if SHOW_STALE_MARKER
      renderSpeedbrakeState(false, true);
```

```
#else
      // Community behaviour: freeze the last valid cockpit indication.
#endif
      break;

    case SYS_TEST:
      // Test mode is handled separately by runTestMode().
      break;

    default:
      renderStandby();
      break;
  }
}

// ---------------------------------------------------------------------
// BENCH TEST MODE
// ---------------------------------------------------------------------

#if USE_TEST_VALUES
void runTestMode() {
  static unsigned long lastTestFrameMs = 0;
  static uint16_t testRaw = 0;
  static int testDirection = 1;
  static bool holdingEndpoint = true;
  static unsigned long endpointHoldStartMs = 0;

  if ((millis() - lastTestFrameMs) < TEST_FRAME_MS) {
    return;
  }

  lastTestFrameMs = millis();

  if (holdingEndpoint) {
    if ((millis() - endpointHoldStartMs) < TEST_ENDPOINT_HOLD_MS) {
      chSpeedbrake.raw = testRaw;
      chSpeedbrake.lastUpdateMs = millis();
```

```
    chSpeedbrake.updateCount++;
    chSpeedbrake.live = true;

    dcsEverSeen = true;
    lastDcsPacketMs = millis();

    currentSbState = decodeSpeedbrakeState(chSpeedbrake.raw);
    renderSpeedbrakeState(true, false);
    return;
  }

  holdingEndpoint = false;
}

if (testDirection > 0) {
  if (testRaw >= (65535 - TEST_OPEN_STEP_RAW)) {
    testRaw = 65535;
    testDirection = -1;
    holdingEndpoint = true;
    endpointHoldStartMs = millis();
  } else {
    testRaw += TEST_OPEN_STEP_RAW;
  }
} else {
  if (testRaw <= TEST_CLOSE_STEP_RAW) {
    testRaw = 0;
    testDirection = 1;
    holdingEndpoint = true;
    endpointHoldStartMs = millis();
  } else {
    testRaw -= TEST_CLOSE_STEP_RAW;
  }
}

chSpeedbrake.raw = testRaw;
chSpeedbrake.lastUpdateMs = millis();
chSpeedbrake.updateCount++;
```

```
  chSpeedbrake.live = true;

  dcsEverSeen = true;
  lastDcsPacketMs = millis();

  currentSbState = decodeSpeedbrakeState(chSpeedbrake.raw);
  renderSpeedbrakeState(true, false);
}
#endif

// ---------------------------------------------------------------------
// OPTIONAL SERIAL DEBUG
// ---------------------------------------------------------------------

void runSerialDebug() {
#if ENABLE_SERIAL_DEBUG
#if !USE_DCS_BIOS
  if ((millis() - lastSerialDebugMs) < SERIAL_DEBUG_MS) {
    return;
  }

  lastSerialDebugMs = millis();

  Serial.print("BUILD=");
  Serial.print(BUILD_NAME);

  Serial.print(" VERSION=");
  Serial.print(BUILD_VERSION);

  Serial.print(" SYS=");
  Serial.print(systemStateName(systemState));

  Serial.print(" SB=");
  Serial.print(speedbrakeStateName(currentSbState));

  Serial.print(" RAW=");
```

```
  Serial.print(chSpeedbrake.raw);

  Serial.print(" TRANSITION=");
  Serial.print(speedbrakeTransitionPercent(chSpeedbrake.raw));
  Serial.print("%");

  Serial.print(" UPDATES=");
  Serial.print(chSpeedbrake.updateCount);

  Serial.println();
#endif
#endif
}

// ------------------------------------------------------------------
// SETUP
// ------------------------------------------------------------------

void setup() {
#if ENABLE_SERIAL_DEBUG
#if !USE_DCS_BIOS
  Serial.begin(115200);
#endif
#endif

  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    for (;;) {
      delay(100);
    }
  }

  display.clearDisplay();
  display.display();

#if SHOW_STARTUP_SPLASH
  systemState = SYS_BOOT;
```

```
  renderStartupSplash();
  delay(STARTUP_SPLASH_MS);
#endif

#if USE_DCS_BIOS
  DcsBios::setup();
#endif

#if USE_TEST_VALUES
  systemState = SYS_TEST;
  chSpeedbrake.raw = 0;
  currentSbState = SB_CLOSED;
  renderSpeedbrakeState(true, false);
#else
  systemState = SYS_STANDBY;
  renderStandby();
#endif
}

// ---------------------------------------------------------------
// LOOP
// ---------------------------------------------------------------

void loop() {
#if USE_DCS_BIOS
  DcsBios::loop();
#endif

#if USE_TEST_VALUES
  runTestMode();
#else
  updateSystemState();
  renderCurrentSystemState();
#endif

  runSerialDebug();
}
```